



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

STUDENT CASE STUDY EXECUTION REPORT

Department	FED
Subject	DSA-2
Academic Year	I-III Semester (2025-26)
Faculty Name	TANGUTURU MADHURI
Student Name	GOTTIPATI EESHA YUKTHA
Roll Number	2520030436
Branch	CSE
Course Outcome	CO2

Case Study Topic: HDFC Net Banking

Work Summary:

In this case study, we constructed the Fenwick (BIT) array for `spend[1..15]`, verified non-trivial BIT cells by tracing the indices they cover, computed the scenario query (Jan 5–12) using `prefix(12) – prefix(4)`, traced the BIT cells visited in each prefix call, identified the bug in the given buggy `prefix_sum` implementation, and argued why BIT is better than a Segment Tree for this specific HDFC workload.

Implementation Details:

1. Initialised BIT array of size 16 (index 0 unused, indices 1–15 active).
2. **BIT Construction:** For each index i with value `spend[i]`, call `pointUpdate(i, spend[i])`. Each update propagates to parent nodes using $i += i \& (-i)$.
3. **Non-trivial cells verified:** `bit[4]` covers indices 1–4, `bit[8]` covers indices 1–8, `bit[12]` covers indices 9–12.
4. **Query Jan 5–12:**
 $\text{prefix}(12) - \text{prefix}(4) = 12050 - 4400 = ₹7,650$. Verified: $1500+600+0+0+3500+0+1100+950 = 7,650 \checkmark$
5. **Bug identified:** $i -= (i \& -i) + 1$ should be $i -= (i \& -i)$. The $+1$ causes incorrect lsb removal, skipping valid BIT nodes and returning wrong prefix sums for many inputs.
6. **BIT vs Segment Tree:** For this workload (point updates only, no range updates), BIT uses n words vs Segment Tree's $\sim 4n$ words, has better sequential cache access, and has far less code — making it ideal for 50M-customer batch processing.

(a) BIT Array Construction & Query Jan 5–12

BIT Array for `spend [1..15]`:

Index	: 1	2	3	4	5	6	7	8
BIT	: 1200	2000	0	4400	1500	2100	0	6500

Index	: 9	10	11	12	13	14	15
BIT	: 3500	3500	1100	5550	700	700	0

Three Non-Trivial BIT Cells Verified:

- **bit[4]:** $\text{lsb}(4) = 4$, covers indices 1–4. $\text{Sum} = 1200 + 800 + 0 + 2400 = 4400 \checkmark$
- **bit[8]:** $\text{lsb}(8) = 8$, covers indices 1–8. $\text{Sum} = 1200 + 800 + 0 + 2400 + 1500 + 600 + 0 + 0 = 6500 \checkmark$
- **bit[12]:** $\text{lsb}(12) = 4$, covers indices 9–12. $\text{Sum} = 3500 + 0 + 1100 + 950 = 5550 \checkmark$

Query: Total Spend Jan 5–12 = $\text{prefix}(12) - \text{prefix}(4)$

```
prefix(12): i=12 → s += bit[12]=5550; i=12-(12&-12)=12-4=8
           i=8 → s += bit[8]=6500; i=8-8=0. Stop.
           prefix(12) = 5550 + 6500 = 12050

prefix(4): i=4 → s += bit[4]=4400; i=4-4=0. Stop.
           prefix(4) = 4400
```

Answer: $12050 - 4400 = \text{Rs. } 7,650$

Verify: $1500 + 600 + 0 + 0 + 3500 + 0 + 1100 + 950 = 7650$ CORRECT

(b) Bug Identification in FenwickBuggy.prefixSum

Buggy Code (given):

```
long prefixSum(int i) {
    long s = 0;
    while (i > 0) {
        s += bit[i];
        i -= (i & -i) + 1; // BUG IS HERE
    }
    return s;
}
```

Bug: The +1 in $i -= (i \& -i) + 1$

- **What it does:** Removes the lowest set bit AND then subtracts 1 extra, causing the traversal to skip valid parent nodes in the BIT.
- **Which inputs trigger it:** Any i where the correct prefix requires more than one BIT node
 ➔ Eg : $\text{prefix}(12)$: correct path is $12 \rightarrow 8 \rightarrow 0$, but buggy code gives $12 \rightarrow (12-4-1) = 7 \rightarrow (7-1-1) = 5 \rightarrow \dots$ — completely wrong.
- **Correct inputs:** Only i that is an exact power of 2 ($i=1,2,4,8,\dots$) where $\text{lsb}(i)=i$, so $i-(i\&-i)=0$ and the loop stops after one step regardless of the +1.
- **One-character fix:** Remove the "+1". Line becomes: $i -= (i \& -i);$
- **Time complexity of buggy version:** $O(\log n)$ — same as correct, but produces WRONG results for most inputs.

Fixed Code:

```
long prefixSum(int i) {
    long s = 0;
    while (i > 0) {
        s += bit[i];
        i -= (i & -i); // FIXED: one-character fix (remove +1)
    }
    return s;
}
```

```

}
return s;
}

```

(c) BIT vs Segment Tree for HDFC Workload

Why BIT is Better for This Specific Workload:

- **(i) Memory:** BIT uses exactly n words (365 integers for a year). Segment Tree uses $\sim 4n$ words (~ 1460 integers). At 50 M customers, BIT saves $\sim 3 \times 365 \times 4 \text{ bytes} \times 50\text{M} = \sim 219 \text{ GB}$ memory in the batch window — critical for feasibility.
- **(ii) Cache Behaviour:** BIT's point-update and prefix-sum access a small, contiguous chain of $\sim \log n$ indices (at most 9 cells for $n=365$). Segment Tree accesses $\sim 2 \log n$ nodes spread across a $\sim 4n$ array with large strides — causing more cache misses during the 50M-customer batch processing.
- **(iii) Code Surface:** BIT is ~ 10 lines of working code vs ~ 40 lines for Segment Tree. Less code = fewer bugs, easier testing, simpler maintenance for a production banking system.

Two Scenarios that Would Flip the Recommendation to Segment Tree:

- **Range Updates Needed:** If HDFC needed to apply bulk adjustments (e.g., add ₹50 cashback to all days in a range), BIT cannot handle range updates efficiently. Segment Tree with lazy propagation handles both range update and range query in $O(\log n)$.
- **Multiple Query Types:** If the system also needed range-minimum, range-maximum, or non-sum aggregates alongside range-sum, a Segment Tree handles all of these generically, whereas BIT only supports invertible prefix operations (sum, XOR).

Result: Screenshots / Output

```

PS C:\Users\Eesha\OneDrive\Documents\DSA_CO2_CASESTUDY> & 'C:\Program Files\Java\jdk-25\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Eesha\AppData\Roaming\Code\User\workspaceStorage\4a72ec84cd095641673eb13619d3bb72\redhat.java\jdt_ws\DSA_CO2_CASESTUDY_c2a825f3\bin' 'FenwickBuggyFixed'
prefixSum(1) = 5
prefixSum(2) = 8
prefixSum(3) = 8
prefixSum(4) = 15
PS C:\Users\Eesha\OneDrive\Documents\DSA_CO2_CASESTUDY>

```

- BIT array constructed for `spend[1..15]`; all 15 cells verified correctly.
 - `bit[4] = 4400`, `bit[8] = 6500`, `bit[12] = 5550` — each traced to confirm coverage.
 - Query Jan 5–12: `prefix(12)–prefix(4) = 12050–4400 = ₹7,650 ✓`
 - Bug: `+1 in i -= (i & -i) + 1`. Fix: remove `+1`. Buggy version correct only for power-of-2 indices.
- Time Complexity:** Point Update $O(\log n)$ | Prefix Sum $O(\log n)$ | Range Query $O(\log n)$. BIT preferred over Segment Tree for this point-update-only, memory-critical HDFC batch workload.

GitHub Link:

https://github.com/egotpat-gif/DSAIL-Case-Studies/tree/main/DSA_CO2_CASESTUDY

Student Signature	Faculty Signature

